тензор (в нашем случае `target`), а не уничтоженный `src_var`.

1. Замените функции обратного прохода в `BASE_CL_FUNCTIONS` (вверху файла) на оптимизированные по выходу:

python

```
_FUNCTIONS = """
float nn_sigmoid(float x) { return 1.0f / (1.0f + exp(-x)); }
float nn_tanh(float x) { return tanh(x); }

ИЗАЦИЯ: принимают y (выход функции), а не x (вход)
float nn_sigmoid_backward_by_y(float y, float go) { return go * y * (1.0f - y); }
float nn_tanh_backward_by_y(float y, float go) { return go * (1.0f - y * y); }
```

Задать вопрос по теме:

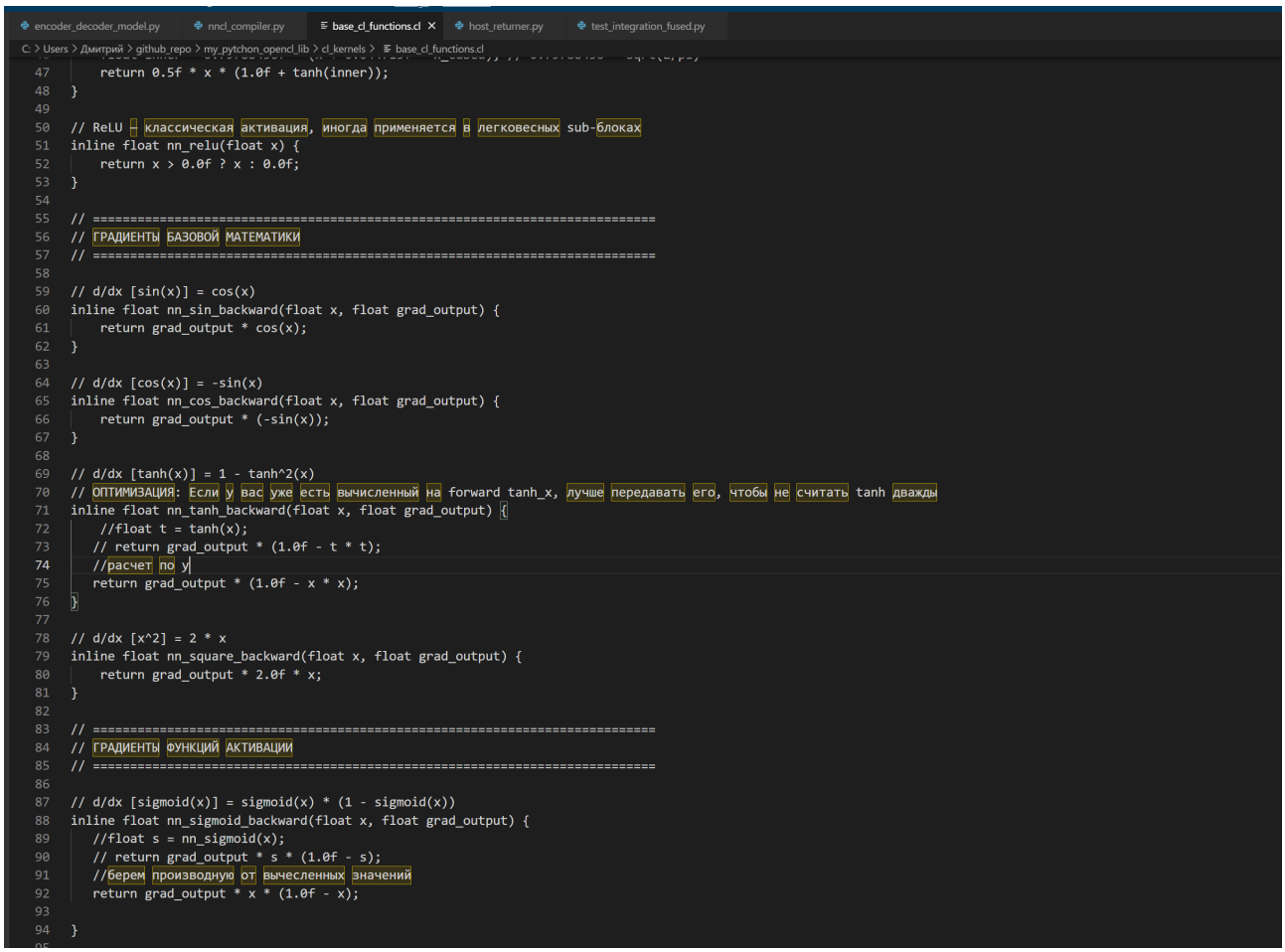
Используйте код с осторожностью.

2. Измените генерацию строк в блоке `# 3. АВТО-BACKWARD` (строки 84–105), передавая туда `target[tid]`:

python

```
                # 3. АВТО-BACKWARD (С использованием оптимизации по Y)
                if self.is_training:
                    if "nn_sigmoid" in cl_op:
                        src_match = re.search(r'nn_sigmoid\((.*?)\)', cl_op)
                        if src_match:
                            src_var = src_match.group(1)
                            clean_src = src_var.replace('[tid]', '')
                            # Передаем target[tid] (текущее значение во VRAM), ма
                            cl_body_backward.append(
                                f"float back_{clean_src} = nn_sigmoid_backward_by
                                f"    grad_{clean_src}[tid] += back_{clean_src};"
```

Я немного запутался с производными.



Тепрпь блок базовых функций должен выглядеть так? Ксатит ,pytorch по идее хранит какото граф вычислений. Не являеться ли этот граф вычислений нужными значениями которых нам нехватает? Если добавить граф вычислений как отдельную переменную в тестирование, решит ли это проблему разных градиентов?